풀 큐 관리 시스템 분석

개념적 배경

geth의 트랜잭션 풀은 두 개의 주요 큐로 구성되어 있습니다: **Pending 큐**와 **Queued 큐**. 이 이중 큐 시스템은 트랜잭션의 실행 가능성에 따라 효율적으로 분류하고 관리하여 블록 생성 시 최적의 트랜잭션 선택을 가능하게 합니다.

큐 시스템이 필요한 이유

1. **실행 순서 보장**: 논스 기반 순차 실행 규칙 준수
2. **성능 최적화**: 즉시 실행 가능한 트랜잭션 빠른 접근
3. **메모리 효율성**: 제한된 리소스 내에서 최대한 많은 트랜잭션 보관
4. **공정성**: 모든 계정에 대한 균등한 기회 제공

핵심 개념

- **Pending 큐**: 즉시 실행 가능한 트랜잭션들 (논스가 연속적)
- **Queued 큐**: 미래에 실행 가능한 트랜잭션들 (논스 갭 존재)
- **Promotion**: Queued에서 Pending으로 이동
- **Demotion**: Pending에서 Queued로 이동

구현 분석

1. 큐 데이터 구조

1.1 SortedMap 구조

위치: `core/txpool/legacypool/list.go:48`

```
type SortedMap struct {
items  map[uint64]*types.Transaction // 논스 -> 트랜잭션 매핑
index *nonceHeap // 논스 힙 인덱스
cache types.Transactions // 정렬된 트랜잭션 캐시
cacheMu sync.Mutex // 캐시 보호 뮤텍스
}
```

특징:

- 논스 기반 빠른 조회 (`O(1)`)
- 힙 기반 정렬된 순회 (`O(log n)` 삽입)
- 캐시를 통한 반복 정렬 비용 절약

1.2 Transaction List 구조

위치: `core/txpool/legacypool/list.go:248`

```
type list struct {
strict  bool  // 논스 연속성 강제 여부
txs *SortedMap // 정렬된 트랜잭션 맵
costcap *uint256.Int // 최고 비용 트랜잭션 가격
gascap  uint64  // 최고 가스 한도
totalcost *uint256.Int // 전체 트랜잭션 비용
}
```

Strict vs Non-Strict 모드:

- **Strict (Pending)**: 논스 갭 허용 안함, 연속성 보장
- **Non-Strict (Queued)**: 논스 갭 허용, 미래 트랜잭션 보관

2. 트랜잭션 추가 및 교체 로직

2.1 트랜잭션 추가 (Add)

위치: `core/txpool/legacypool/list.go:280`

```
func (l *list) Add(tx *types.Transaction, priceBump  uint64) (bool, *types.Trans

// 기존 트랜잭션 확인

old := l.txs.Get(tx.Nonce())

if  old != nil {

// 가격 비교 (Fee Cap과 Tip 모두 확인)

if  old.GasFeeCapCmp(tx) >= 0 || old.GasTipCapCmp(tx) >= 0 {

return false, nil

}

// 가격 상승률 검증

thresholdFeeCap := old.GasFeeCap() * (100 + priceBump) / 100

thresholdTip := old.GasTipCap() * (100 + priceBump) / 100

if  tx.GasFeeCapIntCmp(thresholdFeeCap) < 0 ||

tx.GasTipCapIntCmp(thresholdTip) < 0 {

return false, nil

}

// 기존 트랜잭션 비용 차감
```

```
l.subTotalCost([]*types.Transaction{old})
}
// 새 트랜잭션 비용 추가
cost, overflow := uint256.FromBig(tx.Cost())
if  overflow {
return false, nil
}
l.totalcost.Add(l.totalcost, cost)
// 트랜잭션 저장 및 캐시 업데이트
l.txs.Put(tx)
if  l.costcap.Cmp(cost) < 0 {
l.costcap = cost
}
if  gas := tx.Gas();  l.gascap < gas {
l.gascap = gas
}
return true, old
}
```

2.2 교체 정책

가격 상승률 요구사항:

- 기본값: 10% (PriceBump 설정)
- Fee Cap과 Tip 모두 상승률 조건 만족 필요

- 절대값과 상대값 모두 확인

```
// 예시: 10% 가격 상승 요구
oldPrice := 100 * gwei
newPrice := 110 * gwei  // 10% 증가 필요
```

3. 큐 간 이동 메커니즘

3.1 Promotion (Queued → Pending)

위치: `core/txpool/legacypool/legacypool.go:1448`

```
func (pool *LegacyPool) promoteExecutables(accounts []common.Address) []*types.T
var promoted []*types.Transaction
gasLimit := pool.currentHead.Load().GasLimit
for _, addr := range accounts {
list := pool.queue[addr]
if list == nil {
continue
}
// 1. 오래된 트랜잭션 제거 (낮은 논스)
forwards := list.Forward(pool.currentState.GetNonce(addr))
for _, tx := range forwards {
pool.all.Remove(tx.Hash())
}
```

```
// 2. 비용 초과 트랜잭션 제거

drops, _ := list.Filter(pool.currentState.GetBalance(addr), gasLimit)

for _, tx := range drops {

pool.all.Remove(tx.Hash())

}

// 3. 실행 가능한 트랜잭션 승격

readies := list.Ready(pool.pendingNonces.get(addr))

for _, tx := range readies {

hash := tx.Hash()

if pool.promoteTx(addr, hash, tx) {

promoted = append(promoted, tx)

}

}

// 4. 계정별 큐 크기 제한 적용

caps := list.Cap(int(pool.config.AccountQueue))

for _, tx := range caps {

pool.all.Remove(tx.Hash())

}

}

return promoted

}
```

Promotion 조건:

1. 논스가 현재 계정 논스와 연속적
2. 계정 잔액이 트랜잭션 비용 이상
3. 가스 한도가 블록 가스 한도 이하
4. 논스 갭이 없음

3.2 Demotion (Pending → Queued)

위치: `core/txpool/legacypool/legacypool.go:1635`

```
func (pool *LegacyPool) demoteUnexecutables() {
gasLimit := pool.currentHead.Load().GasLimit
for  addr, list := range  pool.pending {
nonce := pool.currentState.GetNonce(addr)
// 1. 오래된 트랜잭션 제거
olds := list.Forward(nonce)
for  _, tx := range  olds {
pool.all.Remove(tx.Hash())
}
// 2. 비용 초과 트랜잭션 처리
drops, invalids := list.Filter(pool.currentState.GetBalance(addr), gasLimit)
for  _, tx := range  drops {
pool.all.Remove(tx.Hash()) // 완전 제거
}
for  _, tx := range  invalids {
pool.enqueueTx(tx.Hash(), tx, false) // Queued로 이동
```

```
}

// 3. 논스 갭 발생 시 모든 후속 트랜잭션 강등

if  list.Len() > 0 && list.txs.Get(nonce) == nil {

gapped := list.Cap(0)

for  _, tx := range  gapped {

pool.enqueueTx(tx.Hash(), tx, false)

}

}

}

}
```

4. 우선순위 관리 시스템

4.1 Price-based Heap

위치: `core/txpool/legacypool/list.go:580`

```
type pricedList struct {

stales atomic.Int64 // 오래된 가격 포인트 수

all *lookup // 모든 트랜잭션 참조

urgent priceHeap // 긴급 힙 (effective tip 기준)

floating priceHeap // 부유 힙 (gas fee cap 기준)

reheapMu sync.Mutex // 재정렬 뮤텍스

}
```

이중 힙 시스템:

- **Urgent Heap**: Base Fee 기준 Effective Tip으로 정렬
- **Floating Heap**: Gas Fee Cap으로 정렬
- **용량 비율**: Urgent:Floating = 4:1

4.2 트랜잭션 제거 (Discard)

```
func (l *pricedList) Discard(slots  int) (types.Transactions, bool) {
drop := make(types.Transactions, 0, slots)
for  slots > 0 {
// 힙 크기 비율에 따라 제거 대상 선택
if  len(l.urgent.list)*floatingRatio > len(l.floating.list)*urgentRatio {
// Urgent에서 Floating으로 이동
tx := heap.Pop(&l.urgent).(*types.Transaction)
if  l.all.Get(tx.Hash()) != nil {
heap.Push(&l.floating, tx)
}
} else {
// Floating에서 완전 제거
tx := heap.Pop(&l.floating).(*types.Transaction)
if  l.all.Get(tx.Hash()) != nil {
drop = append(drop, tx)
slots -= numSlots(tx)
}
```

```
}
}
return drop, true
}
```

5. 메모리 관리 및 제한

5.1 계정별 제한

```
type Config struct {
AccountSlots uint64 // 계정당 Pending 슬롯 (기본: 16)
AccountQueue uint64 // 계정당 Queued 슬롯 (기본: 64)
GlobalSlots uint64 // 전체 Pending 슬롯 (기본: 5120)
GlobalQueue uint64 // 전체 Queued 슬롯 (기본: 1024)
}
```

5.2 Pending 큐 정리 (truncatePending)

```
func (pool *LegacyPool) truncatePending() {
// 1. 스팸 계정 식별 (AccountSlots 초과)
spammers := prque.New[uint64, common.Address](nil)
for addr, list := range pool.pending {
if length := uint64(list.Len()); length > pool.config.AccountSlots {
spammers.Push(addr, length)
}
```

```
}

// 2. 균등화 알고리즘으로 점진적 제거

for  pending > pool.config.GlobalSlots && !spammers.Empty() {

offender, _ := spammers.Pop()

// 모든 위반자의 트랜잭션 수를 균등하게 조정

threshold := pool.pending[offender].Len()

// ... 균등화 로직

}

}
```

5.3 Queued 큐 정리 (truncateQueue)

```
func (pool *LegacyPool) truncateQueue() {

// 1. 하트비트 기준 계정 정렬 (오래된 순)

addresses := make(addressesByHeartbeat, 0, len(pool.queue))

for  addr := range  pool.queue {

addresses = append(addresses, addressByHeartbeat{

addr, pool.beats[addr]

})

}

sort.Sort(sort.Reverse(addresses))

// 2. 오래된 계정부터 트랜잭션 제거

for  drop := queued - pool.config.GlobalQueue;  drop > 0; {
```

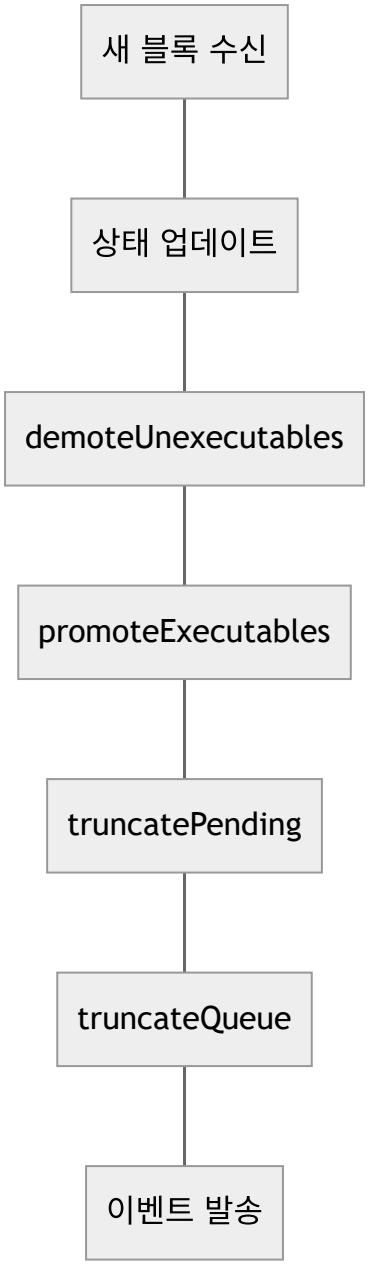
```
addr := addresses[len(addresses)-1]

list := pool.queue[addr.address]

if size := uint64(list.Len()); size <= drop {

// 전체 계정 트랜잭션 제거

for _, tx := range list.Flatten() {

pool.removeTx(tx.Hash(), true, true)

}

drop -= size

} else {

// 일부 트랜잭션만 제거 (높은 논스부터)

txs := list.Flatten()

for i := len(txs) - 1; i >= 0 && drop > 0; i-- {

pool.removeTx(txs[i].Hash(), true, true)

drop--

}

}

}

}
```

실행 흐름

1. 트랜잭션 라이프사이클

```
graph TD

A[트랜잭션 수신] --> B{논스 확인}

B -->|연속적| C[Pending 큐]

B -->|갭 존재| D[Queued 큐]

D --> E{상태 변경}

E -->|논스 갭 해소| F[Promotion]

F --> C

C --> G{블록 포함}

G -->|포함됨| H[풀에서 제거]

G -->|포함 안됨| I{상태 변경}

I -->|잔액 부족| J[Demotion]

J --> D

I -->|논스 갭 발생| J

C --> K{메모리 압박}

K -->|Yes| L[낮은 가격 제거]

D --> M{시간 초과}

M -->|Yes| N[만료 제거]
```

2. 큐 재구성 과정



성능 최적화

1. 캐싱 전략

```
// SortedMap의 정렬 결과 캐싱
func (m *SortedMap) flatten() types.Transactions {
m.cacheMu.Lock()
defer  m.cacheMu.Unlock()
```

```
if m.cache == nil {
m.cache = make(types.Transactions, 0, len(m.items))
for _, tx := range m.items {
m.cache = append(m.cache, tx)
}
sort.Sort(types.TxByNonce(m.cache))
}
return m.cache
}
```

2. 배치 처리

```
// 여러 계정을 한 번에 처리
func (pool *LegacyPool) promoteExecutables(accounts []common.Address) {
// 배치로 처리하여 락 경합 최소화
for _, addr := range accounts {
// 계정별 처리
}
}
```

3. 힙 재구성 최적화

```
// 임계값 기반 재구성
```

```go
func (l *pricedList) Removed(count int) {
stales := l.stales.Add(int64(count))
// 25% 이상 오래된 항목이 있을 때만 재구성
if int(stales) <= (len(l.urgent.list)+len(l.floating.list))/4 {
return
}
l.Reheap()
}
```

실제 사용 예시

1. 트랜잭션 상태 확인

```go
// 트랜잭션 상태 조회
status := pool.Status(txHash)
switch status {
case txpool.TxStatusPending:
fmt.Println("트랜잭션이 Pending 큐에 있음")
case txpool.TxStatusQueued:
fmt.Println("트랜잭션이 Queued 큐에 있음")
case txpool.TxStatusUnknown:
fmt.Println("트랜잭션을 찾을 수 없음")
}
```

2. 큐 내용 조회

```
// Pending과 Queued 트랜잭션 조회
pending, queued := pool.Content()

for addr, txs := range pending {
fmt.Printf("계정 %s: %d개 pending 트랜잭션\n", addr, len(txs))
}

for addr, txs := range queued {
fmt.Printf("계정 %s: %d개 queued 트랜잭션\n", addr, len(txs))
}
```

3. 특정 계정 트랜잭션 조회

```
// 특정 계정의 트랜잭션 조회
pendingTxs, queuedTxs := pool.ContentFrom(address)

fmt.Printf("Pending: %d, Queued: %d\n", len(pendingTxs), len(queuedTxs))
```

에러 처리 및 예외 상황

주요 에러 시나리오

1. **논스 갭 발생**

```
// 논스 2가 없는 상태에서 논스 3 트랜잭션 수신
// → Queued 큐에 보관, 논스 2 도착 시 Promotion
```

2. **잔액 부족**

```
// Pending 트랜잭션이 잔액 부족으로 실행 불가
// → Demotion 또는 제거
```

3. **메모리 압박**

```
// 글로벌 한도 초과 시 낮은 가격 트랜잭션부터 제거
```

복구 메커니즘

```
// 자동 복구 로직
func (pool *LegacyPool) runReorg() {
// 1. 상태 불일치 감지
// 2. 전체 큐 재구성
// 3. 무효한 트랜잭션 제거
// 4. 실행 가능한 트랜잭션 승격
}
```

관련 시스템

의존성 관계

- **상태 데이터베이스**: 계정 논스, 잔액 조회
- **블록체인**: 새 블록 수신 시 큐 재구성
- **P2P 네트워크**: 원격 트랜잭션 수신
- **마이닝**: Pending 큐에서 트랜잭션 선택

상호작용 패턴

1. **블록 수신**: 상태 변경으로 인한 큐 재구성
2. **트랜잭션 수신**: 적절한 큐에 배치
3. **메모리 관리**: 주기적인 정리 작업
4. **이벤트 발송**: 큐 변경 사항 알림

영향 범위

- **블록 생성 속도**: Pending 큐 크기에 따른 트랜잭션 선택 성능
- **메모리 사용량**: 큐 크기 제한을 통한 메모리 관리
- **네트워크 처리량**: 효율적인 큐 관리로 처리량 향상
- **사용자 경험**: 빠른 트랜잭션 확인 및 상태 업데이트

이 큐 관리 시스템은 Ethereum 네트워크의 트랜잭션 처리 효율성과 공정성을 보장하는 핵심 메커니즘으로, 복잡한 상태 관리와 우선순위 결정을 통해 최적의 성능을 제공합니다.